

# Project Report

## Web GUI for an AAS-Based Production Plan System

Author: Prateek Kumar Sharma

April 16, 2026

## 1 INTRODUCTION

---

### 1.1 Background

In modern industrial environments like SmartFactory-KL, production systems rely on **Digital Twins** to monitor physical processes. The **Asset Administration Shell (AAS)** is the key standard here, providing a digital structure for machines and orders. While AAS is excellent for machine-to-machine data exchange, it is not designed for human interaction. The data is stored in complex, nested submodels that are difficult for an operator to read quickly.

### 1.2 What is the Problem?

The main problem is the "Information Gap" between raw AAS data and the people running the factory. Currently, an operator cannot simply look at the raw data and know if a machine is available or which job is running. This data is distributed across different APIs, making it time-consuming to combine. In a live factory, every second counts, and the lack of a clear visual layer makes it hard to identify bottlenecks or scheduling delays.

### 1.3 Let's Define Goal

The goal of this project is to build a professional web-based GUI that acts as a visual bridge for the AAS system. The focus is on creating a tool that supports fast decision-making and clear process monitoring.

- **Data Consolidation:** Bringing scattered AAS submodels into a single timeline.
- **Real-Time Awareness:** Providing instant feedback on the factory's current status.
- **Intuitive Interaction:** Designing a UI that allows users to find details without being overwhelmed by data.

## 2 WHO IS THIS FOR?

---

### 2.1 Primary User Groups

The GUI is built for two specific industrial roles:

- **Industrial Operators:** Users who need to monitor the live status of machines and ensure production is flowing correctly.
- **Production Engineers:** Users who analyze schedules and resource allocation to optimize the factory's efficiency.

### 2.2 User Needs and Design Logic

Both groups require high "Situational Awareness." Their needs guided my design:

- **Reduced Cognitive Load:** Instead of reading numbers, users see colors and shapes to understand status.
- **Time Efficiency:** The interface must answer basic questions (Is machine X running?) in less than three seconds.
- **Consolidated View:** A single page must show the relationship between different machines and time.

### 2.3 Key Use Cases

The GUI supports the following core tasks:

- **Live Status Monitoring:** Seeing which machines are active across the whole factory.
- **Order Tracking:** Identifying which product is currently being processed on which machine.
- **Temporal Analysis:** Reviewing completed work and checking the upcoming schedule.

## 3 HOW I PLANNED IT

---

### 3.1 Conceptual Foundation

My design methodology follows the "Mental Model" approach. Instead of displaying the AAS data structure (which is how a computer sees it), I designed the GUI based on how a human sees it: a sequence of events over time. This reduces the training time needed to use the system.

### 3.2 Design Principles: Why, What, and How

To justify my design decisions, I used three main principles:

1. **Information Hierarchy:** I prioritized the most critical data (Current Time and Production Status) at the top of the visual hierarchy.
2. **Temporal Mapping:** I chose a Gantt-chart style because it is the industry standard for scheduling. Humans naturally process time on a horizontal axis, making it the most logical choice for production planning.
3. **Progressive Disclosure:** I kept the main screen clean and used tooltips for detailed data. This ensures the user isn't "paralyzed" by too much information at once.

### 3.3 Workflow

The project moved from analyzing the BaSyx AAS API to creating structural drafts that separated high-level info from detailed schedules. By testing different visualization methods, I determined that a machine-centric timeline was the most efficient way to communicate production progress.

## 4 BUILDING THE SKELETON

### 4.1 Information Architecture and Page Split

The system is structured into two distinct layers: a **High-Level Dashboard** (Home) (see Figure 1) and a **Detailed Production Plan** (see Figure 2). This separation is a deliberate design choice to manage information density. By decoupling general factory KPIs from the complex machine timeline, we avoid "Information Overload." The operator can first assess the global state of the factory before deciding to "drill down" into specific machine schedules, a pattern known as **Progressive Disclosure**.

### 4.2 Layout Logic and Visual Focus

The interface utilizes a centered, fixed-width layout with intentionally vacant side margins.

- **The "Why":** These vacant sidebars serve as **Negative Space**, which reduces visual noise and draws the user's focus toward the central data core. In industrial monitoring, peripheral distractions can lead to errors; a centered layout ensures the most critical metrics and timeline data remain the primary **Visual Anchor**.
- **Consistency:** This layout remains consistent across both pages, ensuring that the navigation bar and content width do not jump or shift, preserving the user's spatial orientation.



**Figure 1.** Structural draft of the Dashboard illustrating the centered fixed-width layout and use of negative space to anchor visual focus on high-level KPIs.

### 4.3 Navigation and User Flow

I implemented a fixed top navigation bar for immediate access to all system areas. The navigation logic was first validated through structural wireframes (see Figure 2) to ensure a Linear Monitoring Flow.

1. **Status Check:** The user opens the Home page to see the "Factory Info" and "Orders Overview" cards.
2. **Detailed Inspection:** If a delay is spotted, the user switches to the "Production Plan" to see the specific machine at fault.
3. **Context Preservation:** Because the navigation is fixed, the user never loses their place, which is vital for high-speed operational tasks.

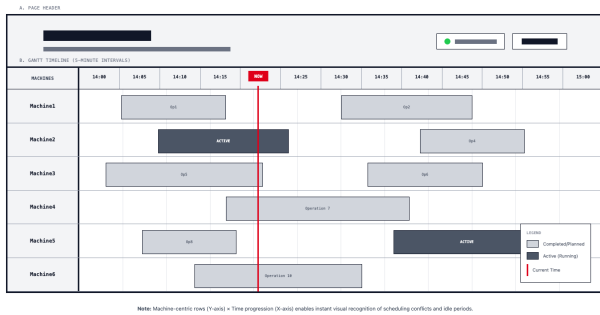
### 4.4 Machine-Centric Structural Decisions

The Production Plan page uses a machine-row structure rather than a table-based list.

- **Logical Reasoning:** Machines are the primary execution units in a factory. Mapping them on the Y-axis and time on the X-axis aligns with the user's **Mental Model** of a physical shop floor.
- **Temporal Relations:** This layout allows the user to see gaps between operations (machine idle time) or overlapping tasks instantly, something a standard text table cannot communicate effectively.

### 4.5 Extensibility and Future-Proofing

The architecture is modular, meaning new views (like "Maintenance" or "Energy Usage") can be added as new tabs in the navigation bar without redesigning the existing core. This ensures that the development effort provides a sustainable foundation that can grow as more AAS submodels become available in the Smart-Factory environment.



**Figure 2.** Conceptual wireframe of the production timeline, highlighting the machine-centric rows (Y-axis) and the high-salience 'NOW' indicator for temporal anchoring.

## 5 VISUALIZATION AND INTERACTION DESIGN

### 5.1 Time as a Horizontal Flow

The central design decision was to implement a Gantt-style timeline rather than a standard data table. This choice is justified by the temporal nature of the AAS production data.

- **The "Why":** Production monitoring is fundamentally about the relationship between time and machine capacity. A horizontal timeline aligns with the user's **Mental Model** of time progression (left-to-right), allowing for an instant understanding of operation sequencing and machine utilization that a static list cannot provide.
- **Spatial Efficiency:** This layout allows for the simultaneous comparison of multiple machines, making it easy to identify idle gaps or scheduling overlaps across the entire factory floor.

### 5.2 Visual Encoding and Color Logic

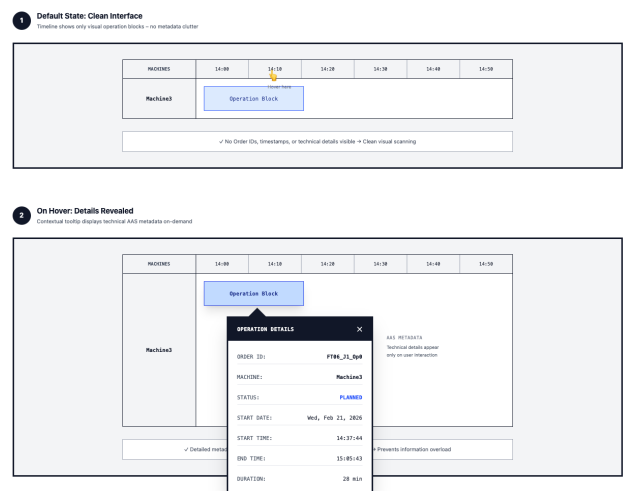
To manage the high density of information, I utilized a multi-layered color strategy:

- **Semantic Grouping:** Operations belonging to the same product order share the same color. This uses the **Gestalt Principle of Similarity**, allowing the eye to track a specific order as it moves across different machines without the user needing to read technical IDs.
- **Visual Hierarchy (Saturation):** Completed operations are de-saturated (greyed out). This "background" historical data, while active and planned operations remain vibrant. This ensures the operator's focus is naturally drawn to relevant, future-oriented tasks.
- **Active Emphasis:** Live operations are given a distinct border highlight, creating **Visual Salience** for the most critical data point: what is happening "right now."

### 5.3 Interaction Design and Progressive Disclosure

To maintain a professional and uncluttered aesthetic, I applied the principle of **Progressive Disclosure** (see Figure 3):

- **Contextual Tooltips:** Technical details (exact durations, order IDs, and AAS references) are hidden by default and appear only on hover. This prevents "Information Paralysis" while keeping deep technical data accessible.
- **Fixed Headers:** While the timeline is scrollable, the machine labels remain fixed on the Y-axis. This preserves **Contextual Awareness**, ensuring the user always knows which machine they are looking at, regardless of how far into the future they scroll.



**Figure 3.** Comparison of the default clean interface vs. the details-on-demand state, illustrating the principle of progressive disclosure to prevent information paralysis.

### 5.4 Real-Time Awareness and Feedback

The system is engineered to provide constant situational feedback through three specific features:

1. **The "NOW" Indicator:** A high-contrast vertical red line acts as a permanent temporal anchor. Its color choice is based on human perception; red is noticed fastest, separating "the past" from "the future" instantly.
2. **Autonomous Follow Mode:** Located in the global control area, this feature keeps the current time centered in the viewport. This is a deliberate design for "hands-free" monitoring, reducing the **Interaction Cost** for the user during live operations.
3. **Synchronized System Clock:** A live clock is positioned at the top right, aligning with modern OS standards to provide a familiar reference point for the user to verify data synchronization.

## 6 USABILITY AND DESIGN PRINCIPLES

### 6.1 Information Filtering and Clarity

The GUI adheres to the principle of **Signal-to-Noise Ratio** optimization. Instead of exposing the entire AAS data model, the interface filters for high-impact variables: machine status, operation timing, and product grouping. This deliberate exclusion of low-priority technical metadata ensures that the user can interpret the state of the factory floor in seconds, preventing the "Information Overload" often associated with raw industrial datasets.

### 6.2 System Consistency and Standards

A rigorous consistency framework was applied to reduce the user's **Cognitive Friction**.

- **Visual Grammar:** We used a unified design language where specific shapes and interaction patterns (like tooltips) behave identically across both the Dashboard and the Production Plan.
- **Predictable Color Logic:** By maintaining the same product-to-color mapping throughout the application, the system supports "Recognition rather than Recall," allowing users to identify production flows without consulting a legend.

### 6.3 Minimizing Cognitive Load

To support the operator's focus, the design minimizes "Extraneous Load" through several structural choices:

- **Page Decoupling:** Separating the global overview from the detailed timeline allows the user to focus on one level of abstraction at a time.
- **Data Visualization over Tabulation:** Converting timestamps into spatial blocks on a timeline allows the human brain to process "durations" and "gaps" as visual patterns rather than mathematical calculations.

### 6.4 Quick Awareness

The final design is optimized for **Rapid Situational Awareness (SA)**. In an industrial environment, the priority is the speed of comprehension.

- **Pre-attentive Processing:** By using the dynamic "NOW" indicator and high-contrast active borders, the GUI leverages pre-attentive visual features, allowing the user to detect live operations almost subconsciously.
- **Feedback Loops:** Real-time polling and the Follow Mode provide a closed-loop feedback system, ensuring the digital representation is always a "Live" reflection of the physical assets.

In summary, the design prioritizes **Operational Speed** over exhaustive data visibility, ensuring the GUI serves as an effective decision-support tool.

## 7 TECHNOLOGY AND IMPLEMENTATION DECISIONS

### 7.1 Frontend Architecture: React for Dynamic Visualization

The frontend was implemented using **React** to handle the high state complexity of the production timeline.

- **High-Frequency Updates:** In a live factory environment, the "NOW" indicator and machine statuses update continuously. React's virtual DOM ensures that only the shifting temporal elements re-render, maintaining 60FPS performance without taxing the client's hardware.
- **Component Modularity:** The UI is built using independent components (Timeline, Card, Tooltip). This ensures that the complex layout of the Gantt chart remains maintainable and that the navigation bar can persist state across page transitions.
- **Separation of Concerns:** React allows the frontend to stay "thin," focusing strictly on user interaction and data presentation while delegating complex logic to the backend.

### 7.2 Backend Logic: FastAPI as a High-Performance Bridge

**FastAPI** was chosen as the middleware layer between the raw data and the UI.

- **Asynchronous Processing:** Since the system polls external AAS servers frequently, FastAPI's asynchronous capabilities prevent the API from "blocking" during data retrieval, ensuring a smooth flow of information.
- **Schema Validation:** Using Pydantic models ensures that the frontend always receives clean, validated data. If an AAS submodel is missing a field, the backend handles the error gracefully rather than breaking the user interface.

### 7.3 Data Source and Transformation Layer

The system consumes data from the **Eclipse BaSyx AAS server**. However, raw AAS submodels are designed for semantic storage, not for visualization.

- **The Semantic Gap:** AAS data is nested and distributed. A "Machine" submodel does not inherently know which "Operation" is currently running on it without cross-referencing.

- **Transformation Logic:** I implemented a dedicated transformation layer in the backend. This layer performs Data Flattening: it groups operations by machine ID, maps timestamps to duration intervals, and assigns consistent color identifiers to product orders.
- **Frontend Optimization:** By performing this heavy computation on the server, the frontend receives a "visualization-ready" JSON object, which significantly reduces the browser's processing load.

overwhelming the user. The two-tier information architecture ensures that both global KPIs and granular machine schedules are accessible within a single, consistent user flow.

Ultimately, this project demonstrates a scalable and modular approach to visualizing Digital Twin data. The resulting application not only meets the current requirements for production monitoring but also provides a stable, extensible foundation for future industrial dashboard modules.

## 7.4 Integrated System Architecture

The system follows a strict **Layered Architecture**. This structure is a deliberate safety decision:

1. **Data Layer (BaSyx):** Acts as the persistent storage of the Digital Twin.
2. **Logic Layer (FastAPI):** Acts as a translator, converting technical AAS schemas into human-oriented production objects.
3. **Presentation Layer (React):** Acts as the interactive portal for the operator.

This decoupling ensures System Resilience. If the AAS server becomes temporarily unresponsive, the backend can serve cached data or clear error states, preventing the GUI from crashing, a requirement for reliable industrial monitoring tools.

## 8 CONCLUSION

---

This project successfully implemented a web-based monitoring interface that bridges the gap between complex AAS data structures and human-centric industrial operations. By transforming distributed sub-models into a centralized, time-oriented visualization, the system provides a functional and intuitive tool for monitoring the SmartFactory-KL ecosystem.

The core of the project's success lies in the deliberate separation of concerns within its architecture. The **FastAPI backend** serves as a critical transformation layer, resolving the "Semantic Gap" of raw AAS data by flattening it into a visualization-ready format. Meanwhile, the **React frontend** provides a high-performance environment capable of handling live updates and complex interactions like "Follow Mode" and dynamic tooltips.

From a design perspective, the implementation proves that industrial GUIs can be both technically deep and user-friendly. By applying UI/UX principles, such as **Progressive Disclosure** for technical details and **Visual Salience** for the live "NOW" indicator, the interface supports rapid situational awareness without